

# HyperHex: Four-Dimensional Hexagonal Corridors for Low-Altitude Drone Traffic

Naresh Yegireddi  
Independent Researcher  
Little Elm, TX, USA  
yegireddi.naresh@gmail.com

**Abstract.** Delivery, inspection and emergency drones will soon share the same low-altitude city corridors, and planning them one at a time with static geofences does not scale. We present HyperHex, a four-dimensional corridor reservation system for low-altitude drone traffic. HyperHex divides the space above a city into hexagonal prisms formed by H3 cells and stacked altitude layers, and adds time as the fourth dimension. A central coordinator reserves each prism, and each move between prisms, for a time interval before a drone is cleared to use it. The corridor graph is never built in full: neighbors are generated only during search, and a sparse ledger stores only the intervals that drones actually hold. Routes are found with a safe-interval variant of A\* and committed atomically, and a temporary no-fly zone triggers local replanning of only the affected drones. We built an open simulator with a CesiumJS 3D workbench and compared HyperHex with a static geofence controller and a reactive hover controller. Over 180 randomized runs with 8 to 48 concurrent drones, HyperHex recorded zero losses of separation, while the baselines recorded up to 8.9 and 4.6 per run. Planning cost was 0.25 to 0.36 ms per drone, and the ledger grew linearly with traffic (about 8.4 intervals per drone).

**Index Terms:** Unmanned aircraft system traffic management (UTM), low-altitude drone corridors, H3, discrete global grid systems, safe interval path planning, 4D trajectory reservation, multi-agent path finding.

## I. Introduction

Package delivery, infrastructure inspection and emergency response are pushing drone operations into dense urban settings. The useful flying band in a city is narrow: drones generally operate a few tens of meters to roughly 150 m above ground, between rooftops and crewed aviation. As traffic grows, this low-altitude band turns from a mostly empty space into a shared resource that must be organized. Current concepts for Unmanned Aircraft System Traffic Management (UTM) rely mainly on sharing flight intent, static geofences and onboard detect-and-avoid [1], [2]. These mechanisms say where drones may not fly, but they do not decide which drone may use which corridor segment at which time.

A natural answer is to treat the low-altitude band as a set of corridors that are booked in advance, much like a reservation system. The difficulty is size. A metropolitan region of roughly  $2 \times 10^5$  surface cells, 20 altitude layers and  $10^4$  time steps gives about  $4 \times 10^{10}$  possible states, nearly all of which stay unused. Any method that enumerates this space is impractical.

HyperHex combines three ideas. First, the ground is indexed with the H3 hexagonal hierarchical grid [3], whose uniform six-neighbor adjacency avoids the diagonal ambiguity of square grids. Stacking altitude layers on each hexagon gives a column of prisms, and each prism over time becomes a 4D corridor

unit. Second, the 4D graph is never constructed. Transitions are generated on demand during search, so empty corridors cost no memory. Third, a central coordinator lets drones reserve time intervals on prisms and on prism-to-prism moves before using them, and the planner searches over safe intervals [4] rather than individual time ticks. Because every reservation passes through one ledger, the same structure also gives operators a single place to monitor the corridor network.

Our contributions are: (i) a formal 4D hexagonal corridor model with node and edge reservation semantics; (ii) a lazy SIPP-style planner integrated with a sparse interval ledger and bounded local repair; (iii) an open simulator and 3D workbench; and (iv) an empirical comparison with two baseline controllers under increasing drone density.

## II. Related Work

*Discrete global grids.* Hexagonal discrete global grid systems offer near-uniform cell area and equidistant neighbors [5]. Uber’s H3 [3] provides 16 resolutions with hierarchical parent and child relations and is widely used in geospatial analytics. Its use as the basic unit of a drone corridor network is less explored.

*Multi-agent path finding.* Cooperative A\* [6] reserves space-time cells in a reservation table. Conflict-Based Search [7] offers optimality at higher cost. Safe Interval Path Planning (SIPP) [4] groups contiguous free time into intervals, which greatly reduces the number of search states in dynamic environments.

*UTM architectures.* NASA’s UTM concept of operations [1] and the European U-space concept [2] define services for strategic deconfliction, while geofencing and tactical avoidance remain the main operational safety layers. HyperHex is positioned as a strategic corridor reservation layer with bounded tactical repair.

## III. System Model

### A. Corridor Units

Let  $\mathcal{H}_r$  be the set of H3 cells at resolution  $r$ ,  $\mathcal{Z} = \{z_0, \dots, z_{K-1}\}$  a set of altitude layers of thickness  $\Delta z$ , and  $\mathcal{T} \subset \mathbb{R}_{\geq 0}$  time. A corridor unit (voxel) is

$$v = (h, k) \in \mathcal{H}_r \times \{0, \dots, K-1\}, \quad (1)$$

and a reservation is a tuple  $\rho = (v, [t_a, t_b], i)$  that grants drone  $i$  exclusive use of  $v$  over the interval. Time is deliberately left out of the key and appears only in the interval.

### B. Lazy Transition Model

From state  $(h, k, t)$  a drone may move horizontally to  $h' \in N_1(h)$  (the six ring-1 neighbors), change layer to  $k \pm 1$ , or hold. The successor set

$$\text{succ}(h, k) = \{(h', k) \mid h' \in N_1(h)\} \cup \{(h, k \pm 1)\} \cup \{(h, k)\} \quad (2)$$

has at most nine elements and is generated only during expansion; no global adjacency is stored. Transitions that exceed vehicle limits (horizontal speed, climb and descent rates) are pruned.

### C. Sparse Interval Ledger

The ledger maps a voxel key to a sorted list of occupied intervals,

$$\mathcal{L}(v) = \left\{ \left[ t_a^{(j)}, t_b^{(j)} \right] \right\}_j, \quad (3)$$

and a missing key means the voxel is free at all times. The safe intervals of  $v$  are the complement of  $\mathcal{L}(v)$  on  $[0, \infty)$ . A second map stores reservations on undirected edges  $e = \{v, v'\}$ , so a move  $v \rightarrow v'$  conflicts with  $v' \rightarrow v$  at overlapping times. This prevents the head-on swap collision that node reservations alone cannot detect.

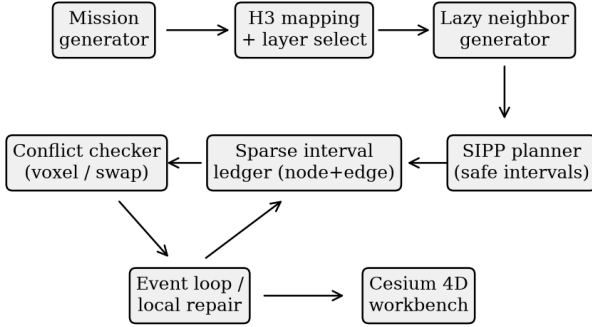


Figure 1: HyperHex architecture. Missions are mapped to H3 corridor units, planned lazily over safe intervals, committed to the node and edge ledger, and replayed in the 4D workbench.

## IV. Planning and Coordination

### A. Safe-Interval Search

The planner (Algorithm 1) runs A\* over states  $(v, \iota)$ , where  $\iota$  indexes a safe interval of  $v$ . The heuristic is the H3 grid distance to the goal divided by the maximum cells per slot. It is admissible because each horizontal move covers at most one cell. When an arrival time falls inside an occupied interval, the search waits at the predecessor until the next safe interval opens, provided the waiting voxel stays free and the connecting edge is unreserved.

### Algorithm 1: Lazy SIPP over the hexagonal ledger

**Input:** start  $(v_0, t_0)$ , goal cell  $g$ , ledger  $\mathcal{L}$ , no-fly set  $F$   
1: OPEN  $\leftarrow \{(v_0, t_0)\}$  ordered by  $f = t + h(v)$   
2: **while** OPEN not empty **do**  
3:  $(v, t) \leftarrow \text{pop min } f$ ; **if**  $v.h = g$  **return** path  
4: **for**  $v' \in \text{succ}(v)$ ,  $v'.h \notin F$  **do**  
5:   **for** each reachable safe interval  $\iota$  of  $v'$  **do**  
6:      $t' \leftarrow \text{earliest } t'' \geq t + 1$  in  $\iota$  with  $v$  free on  $[t, t'']$   
7:     **if** edge  $(v, v')$  free at  $t' - 1$  and  $(v', \iota)$  improved  
8:       push  $(v', t')$   
9: **return** fail (hold on ground)

### B. Commit, Release and Repair

A successful path is committed by merging consecutive same-voxel states into one interval and adding one edge interval per move. When a temporary no-fly zone closes a set of cells  $F$  at time  $t_f$ , only drones whose future reservations intersect  $F$  are affected. For each of them, the ledger truncates its intervals at  $t_f$  and the planner runs again from the current state. All other drones keep their reservations unchanged. This bounded repair contrasts with global replanning and keeps disruption local.

### C. Baseline Controllers

*Static geofence* plans the shortest feasible route that avoids  $F$  but does not coordinate between drones. *Reactive* follows its shortest route and hovers whenever the next voxel is occupied at the next slot, approximating purely tactical avoidance without reservations.

## V. Experimental Setup

The simulator is written in TypeScript with h3-js and runs headlessly; the same code drives the CesiumJS workbench (Fig. 6). Table I lists the parameters. Downtown San Francisco (37.7793° N, 122.4193° W) is the center of a radius-6 H3 disk at resolution 9 (127 cells, each roughly 0.1 km<sup>2</sup> in area), with three altitude layers.

Table I: Simulation parameters

| Parameter                  | Value                         |
|----------------------------|-------------------------------|
| H3 resolution / region     | 9 / radius-6 disk (127 cells) |
| Altitude layers            | 60, 100, 140 m (30 m thick)   |
| Time slot / horizon        | 4 s / 90 slots                |
| Moves per slot             | 1 cell, 1 layer, or hold      |
| Departure window           | uniform, slots 0 to 29        |
| Min. origin to destination | 5 cells                       |
| Drone counts $N$           | 8, 16, 24, 32, 40, 48         |
| Seeds per level            | 10 (fixed LCG seeds)          |
| Conflict definition        | same voxel and slot, or swap  |

Two experiments were run. E1 (scenario): a curated 15-drone scenario (12 radial flights plus a three-drone vertical stack) with a radius-1 temporary no-fly zone activated at slot 8. E2 (density sweep): for each  $N$  and seed, random missions with random layers were admitted under each controller, giving 180 runs. The metrics are losses of separation, completed missions, delay

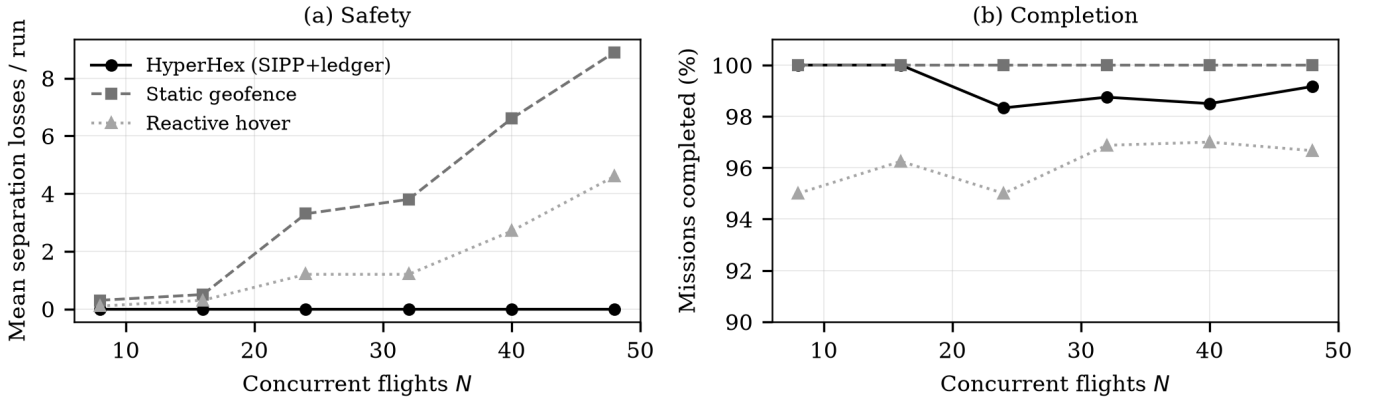


Figure 2: E2 density sweep (10 seeds per point). (a) Mean losses of separation per run. (b) Fraction of missions reaching their destination within the horizon.

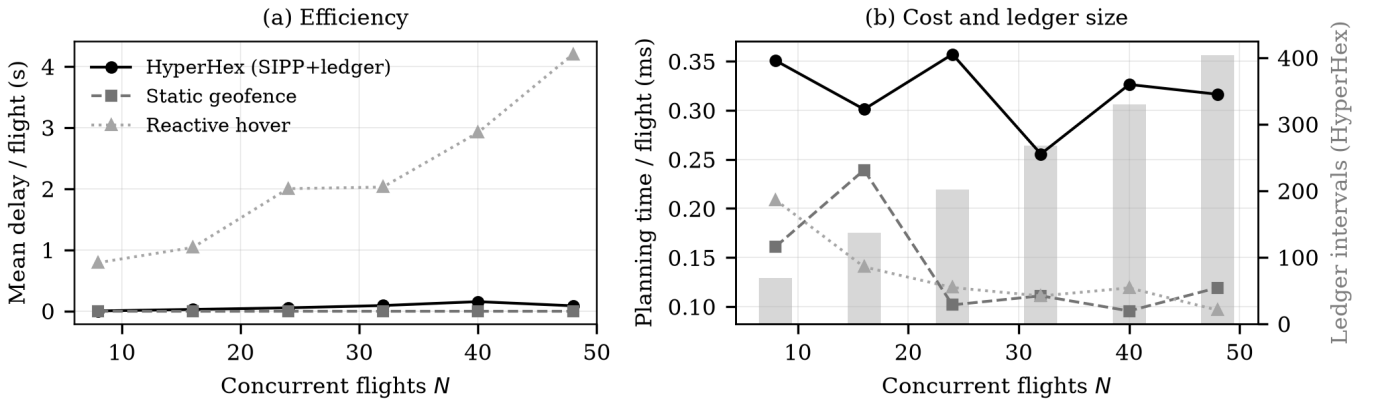


Figure 3: E2 efficiency. (a) Mean delay per completed drone relative to the free-flight bound. (b) Planning wall-clock time per drone (lines) and HyperHex ledger size (bars).

beyond the free-flight lower bound (grid distance), wall-clock planning time and ledger size. Delays are reported in seconds (slots  $\times 4$  s).

## VI. Results

### A. E1: Curated Scenario with a No-Fly Zone

Table II summarizes E1. Before the no-fly zone appeared, the geofence controller already showed one loss of separation; after it, six. HyperHex replanned the same six affected drones with zero conflicts and 12 s of total added delay. The reactive controller also avoided conflicts but accrued 48 s of delay and failed to complete one mission within the horizon.

Table II: E1: 15 drones, no-fly zone at slot 8

| Controller      | Conflicts | Delay (s) | Done  | Replans |
|-----------------|-----------|-----------|-------|---------|
| HyperHex        | 0         | 12        | 15/15 | 6       |
| Static geofence | 6         | 8         | 15/15 | 6       |
| Reactive        | 0         | 48        | 14/15 | 6       |

The HyperHex ledger held 195 intervals after repair, compared with  $127 \times 3 \times 90 = 34,290$  voxel-slots in the equivalent dense table (0.57%).

### B. E2: Density Sweep

Fig. 2 shows that losses of separation grow roughly linearly with  $N$  for both baselines (geofence from 0.3 to 8.9, reactive from 0.1 to 4.6 per run), while HyperHex stayed at zero across all 180 runs. HyperHex completed 98.3% to 100% of missions. The remaining 0.4 to 0.6 missions per run are drones for which no conflict-free route existed within the horizon; they were held on the ground rather than launched into conflict. The geofence controller completes every mission precisely because it ignores other traffic.

Fig. 3(a) shows that this safety comes at a small delay: at most 0.15 s per completed drone on average, against up to 4.2 s for the reactive controller at  $N = 48$ . Planning time stayed between 0.25 and 0.36 ms per drone with no trend in  $N$ , and the ledger grew linearly from 68 to 404 intervals.

### C. Storage Scaling

Fig. 4 compares stored state counts. Because only occupied intervals are kept, storage depends on traffic rather than on the size of the corridor network. This is the property that makes city-scale deployment plausible.

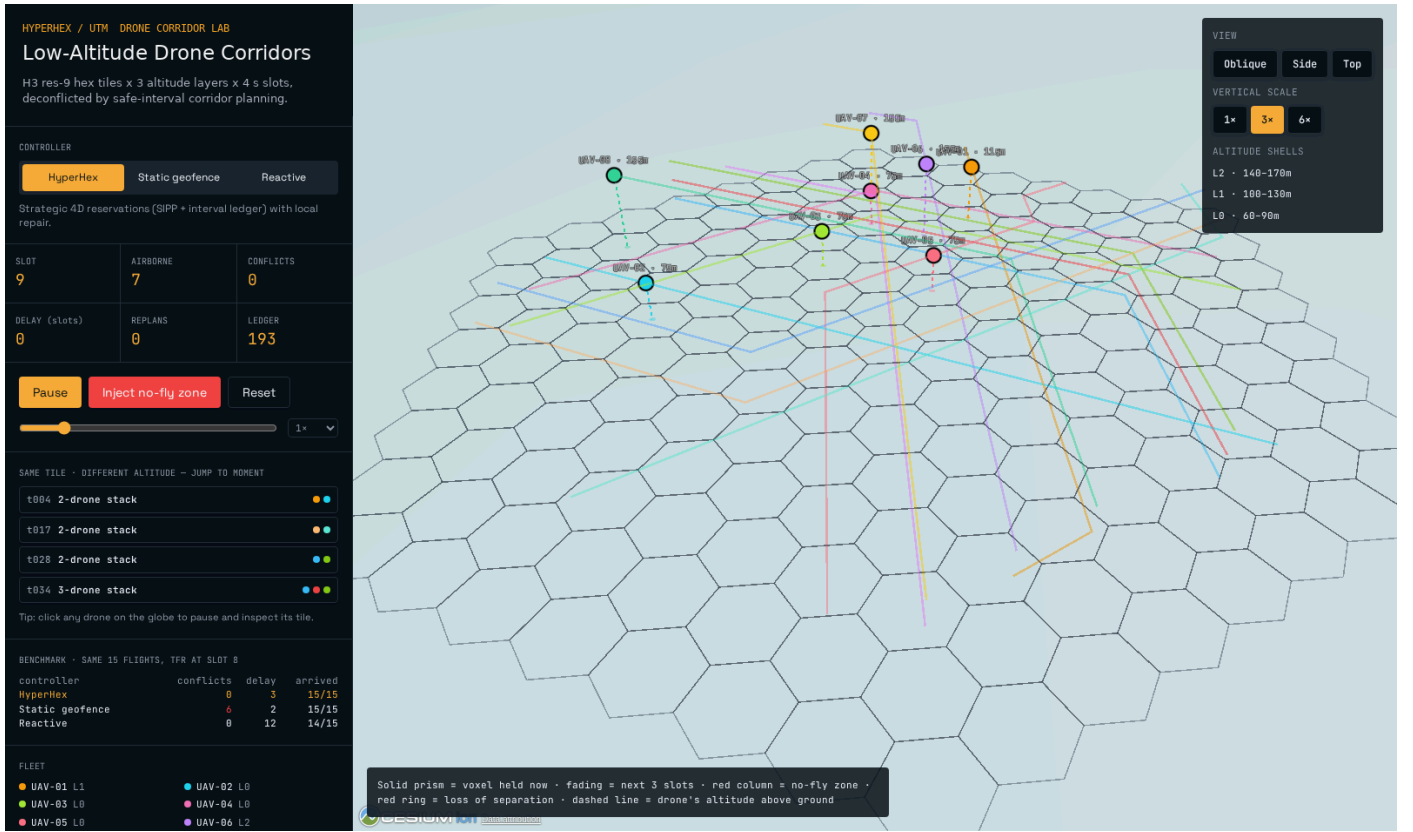


Figure 6: HyperHex CesiumJS workbench (3× vertical exaggeration): controller selection, live metrics, same-tile and different-altitude events, altitude stalks and H3 tiling.

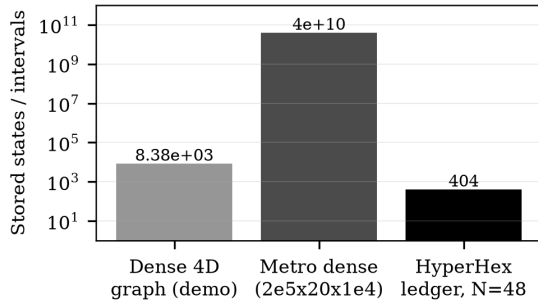


Figure 4: Stored states (log scale): dense 4D table for the demo region, dense table for a metropolitan region, and the HyperHex ledger at  $N = 48$ .

## VII. Case Study: Vertical Separation in a Shared Tile

In a top-down map, drones crossing the same hexagon appear to collide. HyperHex allows several drones in one horizontal cell as long as their altitude layers differ. In E1 the simulator found 24 distinct (cell, slot) events in which at least two drones shared a tile on different layers, including one with three drones at 60, 100 and 140 m (Fig. 5). The workbench (Fig. 6) lets an operator pause on such an event and inspect the column of corridor units.

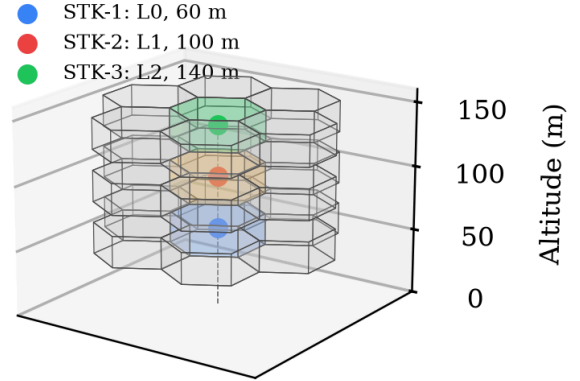


Figure 5: Three drones occupy the same H3 tile in the same slot on layers L0 to L2 (40 m vertical spacing); neighboring prisms shown in grey.

## VIII. Discussion and Limitations

This evaluation is a small-scale simulation and should be read as a proof of concept. Time is discretized into 4 s slots, and the planner applies the safe-interval idea on top of this discretization rather than over fully continuous time. Vehicle dynamics are reduced to one move per slot; energy, wind and mixed fleets are not modeled. Separation is defined at voxel granularity without protected-neighborhood buffers. The ledger is in memory with a single writer, so distributed atomic commit is not evaluated. Finally, delay is measured against a grid-distance lower bound rather than an optimal multi-agent solution.

## IX. Conclusion and Future Work

HyperHex shows that hexagonal hierarchical indexing, lazy graph expansion and a sparse node and edge interval ledger together form a compact, conflict-free reservation layer for low-altitude drone corridors. Because all reservations pass through one ledger, the same layer can serve as a central point for monitoring and managing drone traffic. In simulation it eliminated the losses of separation that grew linearly under uncoordinated and reactive control, at sub-millisecond planning cost and with storage proportional to traffic. Future work includes continuous-time SIPP with kinematic profiles, multi-resolution corridor planning using H3 parent cells, protected neighborhoods, priority and fairness policies, distributed ledgers with atomic commit, weather-driven dynamic closures, and validation on recorded flight telemetry.

## References

- [1] P. Kopardekar, J. Rios, T. Prevot, M. Johnson, J. Jung, and J. E. Robinson, “Unmanned Aircraft System Traffic Management (UTM) Concept of Operations,” technical report, 2016.
- [2] SESAR Joint Undertaking, “U-space Concept of Operations, Edition 03.10,” technical report, 2023.
- [3] Uber Technologies, “H3: A Hexagonal Hierarchical Geospatial Indexing System,” 2018.
- [4] M. Phillips and M. Likhachev, “SIPP: Safe Interval Path Planning for Dynamic Environments,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2011, pp. 5628-5635.
- [5] K. Sahr, D. White, and A. J. Kimerling, “Geodesic Discrete Global Grid Systems,” *Cartography and Geographic Information Science*, vol. 30, no. 2, pp. 121-134, 2003.
- [6] D. Silver, “Cooperative Pathfinding,” in *Proc. AAAI Conf. Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2005, pp. 117-122.
- [7] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, “Conflict-Based Search for Optimal Multi-Agent Pathfinding,” *Artificial Intelligence*, vol. 219, pp. 40-66, 2015.